MLE-STAR Reconstruction Plan

A guide for returning teammates

This document explains what MLE-STAR is, why we are rebuilding it, and exactly what needs to be implemented. It is written so that someone with no prior project context can pick it up and start working.

Branch: feature/reconstruct-mle-star-openai

Base branch: feature/issue-1.5-openai-port

Generated: 2026-06-16

1. What is MLE-STAR?

MLE-STAR is an autonomous multi-agent system for machine-learning engineering. In plain language: you give it a dataset and a task description, and a team of artificial-intelligence agents designs, trains, debugs, and improves a machine-learning solution for you.

Imagine a small data-science team where each member has a specialty: one researches which models work well, another writes the first training script, another tries removing parts of the code to see what matters most, another combines several models into an ensemble, and a final member prepares the submission file. MLE-STAR automates that entire team.

1.1 The agents and their roles

- Task summarizer: reads the competition or task description and distils it.
- Model retriever: proposes which machine-learning models are worth trying.
- Model evaluator: writes a first training script and measures its validation score.
- Merger: combines two candidate solutions into a stronger one.
- Data-use checker: makes sure the solution uses all the information provided.
- Refinement agent: runs ablation studies to find the most important parts of the code and improves them.
- Ensemble agent: combines the best refined solutions into a single stronger model.
- Submission agent: generates the final script that trains on all training data and writes submission.csv.

1.2 What makes it powerful

The system does not just try one model. It generates several candidates, merges them, studies which parts of the code actually improve the score, refines those parts, builds an ensemble, and finally produces a submission. This iterative, multi-agent approach is what makes MLE-STAR competitive on tabular machine-learning competitions.

2. Why are we rebuilding it?

The original MLE-STAR was built on top of Google ADK and Gemini. ADK is a framework that orchestrates agents, and Gemini is the large language model that powers them. We cannot use those services in our current environment, so we need a version that runs on the university's OpenAI-compatible API.

2.1 What already works

A teammate already created a minimal OpenAI-compatible port (feature/issue-1.5-openai-port). It can:

- Connect to the university API.
- Run a very simplified two-step pipeline: propose one model, generate one training script, and create a submission file.
- Execute generated Python code safely and parse the validation score.

2.2 What is missing

That minimal port does not include the full MLE-STAR agent architecture. In particular, it is missing:

- Multiple model candidates and the merging step.
- The ablation-based refinement loop.
- The ensemble agent.
- The data-leakage and data-usage checkers.

2.3 Goal of this rebuild

Our goal is to port every agent and every loop from the original MLE-STAR so that the full pipeline runs end-to-end using only the university API. Once that is stable, we will integrate skrub and MCTS as a later improvement.

3. Current state vs. original MLE-STAR

3.1 Current minimal port

The existing port executes this short pipeline:

```
run_pipeline()
  ├── prepare_task()
  ├── create_workspace(task_id=1)
  ├── task_summarization_agent
  ├── model_retriever_agent (1 candidate, no web search)
  ├── model_eval_and_debug (1 candidate)
  ├── rank_candidate_solutions
  ├── select_best_solution
  └── submission_agent
```

3.2 Original full pipeline

The original MLE-STAR pipeline looks like this:

```
root_agent
  ├── initialization_agent
  │     ├── task_summarization_agent
  │     └── init_parallel_agent (per task_id)
  │           ├── model_retriever_agent (Google Search)
  │           ├── model_eval_and_debug_loop (M candidates)
  │           ├── merger_and_debug_loop
  │           ├── selection_agent
  │           └── check_data_use_and_debug_loop
  ├── refinement_agent (per task_id)
  │     └── ablation_and_refine_loop
  │           ├── ablation_and_debug_loop
  │           ├── ablation_summary_agent
  │           ├── init_plan_agent
  │           ├── init_plan_implement_agent
  │           └── refine_inner_loop
  ├── ensemble_agent
  │     ├── init_ensemble_plan_agent
  │     ├── init_ensemble_plan_implement_agent
  │     └── ensemble_plan_refine_and_implement_loop
  └── submission_agent
```

Note: The main difference is depth. The minimal port only proposes one model and submits it. The original system explores, merges, studies, refines, and ensembles before submitting.

4. Glossary

Agent

An LLM-powered step that receives an instruction, optionally calls the language model, and produces an output (text, code, or a decision).

OpenAI-compatible API

The university's language-model endpoint. It accepts the same request format as OpenAI's API, so we can use standard OpenAI client libraries.

ADK

Google's Agent Development Kit, the framework used by the original MLE-STAR. We are replacing it with our own runner.

Runner

Our custom Python module that executes agents sequentially or in loops, tracks state, and records token usage.

AgentState

A dictionary that holds all information shared between agents during a pipeline run.

Model candidate

One proposed machine-learning model or approach, for example GradientBoostingRegressor or a neural network.

Ablation study

An experiment where you remove or change one part of the code at a time to measure how much it affects performance.

Ensemble

Combining predictions from multiple models to get a better final prediction.

Data leakage

Using information from the test set or future during training, which makes the validation score unrealistically good.

Data usage checker

An agent that verifies the solution uses all the information provided in the task description.

Hold-out validation

Splitting the training data into train and validation sets to estimate performance without touching the test set.

skrub

A Python library for building tabular ML pipelines with declarative choices. It will be integrated in a later phase.

MCTS

Monte Carlo Tree Search, a search algorithm that will later be used to explore skrub pipeline configurations efficiently.

5. Project decisions

These decisions have already been made and should be respected while implementing:

- Base branch: feature/issue-1.5-openai-port
- New branch: feature/reconstruct-mle-star-openai
- Language model: university OpenAI-compatible API (meta-llama-3.1-8b-instruct for testing)
- Web search: disabled in this phase; will be added later
- Parallelism: code must support multiple solutions, but we start with num_solutions=1
- Data leakage checker: enabled
- Data usage checker: enabled
- Priority: make the full pipeline run end-to-end; competitive scores come later
- Prompts: keep the original MLE-STAR prompts whenever possible

5.1 Initial configuration

```
num_solutions: 1
num_model_candidates: 2
max_retry: 2
max_debug_round: 2
max_rollback_round: 2
inner_loop_round: 1
outer_loop_round: 1
ensemble_loop_round: 1
use_data_leakage_checker: True
use_data_usage_checker: True
```

6. Implementation phases

The rebuild is organised in six phases. Phases 0-5 must be completed before we add skrub and MCTS. Each phase builds on the previous one, so it is important to finish one phase before moving to the next.

Phase 0: Preparation

- Create branch feature/reconstruct-mle-star-openai from feature/issue-1.5-openai-port.
- Verify environment: pytest passes, API is reachable.
- Update config.py with conservative values for the first end-to-end run.

Phase 1: Strengthen initialization

- Allow the model retriever to propose more than one candidate.
- Run and debug every candidate.
- Implement the merger that combines the best two candidates.
- Implement the selection step that promotes the merged solution.
- Implement the data-use checker.
- Integrate the data-leakage checker into every execution step.

Phase 2: Implement refinement

- Create the refinement agent and its prompts.
- Port the ablation agent, summary agent, and planning agents.
- Wire the inner and outer refinement loops.
- Integrate the data-leakage checker.

Phase 3: Implement ensemble

- Create the ensemble agent and its prompts.
- Port the ensemble planning and implementation agents.
- Wire the ensemble refinement loop and integrate leakage checks.

Phase 4: Strengthen submission

- Update the submission agent to choose the best refined or ensemble solution.
- Ensure final_solution.py trains on all data and writes submission.csv.

Phase 5: Integration and validation

- Connect all phases in the top-level agent.py.
- Run the full pipeline on California Housing.
- Fix errors until the pipeline finishes end-to-end.

7. Task board — GitHub Issues / Jira cards

The following cards can be imported directly into a GitHub Project board or a Jira backlog. Each card has a unique ID, a clear title, the phase it belongs to, a short description, and concrete acceptance criteria.

ID	Title	Phase
R-00	Create reconstruction branch	Phase 0

Description:
Create feature/reconstruct-mle-star-openai from feature/issue-1.5-openai-port.

Accep

Branch exists and is checked out; no uncommitted conflicts.

R-01	Verify environment and baseline tests	Phase 0

Description:
Run existing pytest suite and confirm API connectivity.

Accep

All existing tests pass; a sample LLM call returns a response.

R-02	Set conservative config defaults	Phase 0

Description:
Update config.py with values suitable for the first end-to-end run.

Accep

Config reflects num_solutions=1, num_model_candidates=2, loops=1, checkers=True.

R-03	Extend model retriever for multiple candidates	Phase 1

Description:
Adapt model_retriever_agent to propose num_model_candidates models and robustly parse the response.

Accep

Parsing succeeds for 1 and 2 candidates; model descriptions stored per task_id/model_id.

R-04	Implement model eval loop for all candidates	Phase 1

Description:
Run and debug every model candidate, storing each init_code and exec_result.

Accep

All candidates are evaluated; scores are parsed and ranked.

R-05	Implement merger agent	Phase 1

Description:
Create merger_and_debug_loop that integrates base_solution with each reference_solution.

Accep

Merger generates executable ensemble code; debug loop fixes errors.

R-06	Implement merger state update	Phase 1

Description:
Track best_score, base_solution, and best_idx after each merge.

Accep

State keys best_score_{task_id}, base_solution_{task_id}, best_idx_{task_id} are correct.

R-07	Implement selection agent	Phase 1

Description:
Promote the best merged solution to train_code_0_{task_id} and write train0.py.

Accep

train0.py exists and matches the selected code.

R-08	Implement check data use agent	Phase 1

Description:
Create check_data_use_and_debug_loop that verifies all provided information is used.

Accep

Agent runs and updates code when information is missing.

R-09	Integrate data leakage checker into initialization	Phase 1

Description:
Attach leakage checks to model_eval, merger, and check_data_use executions.

Accep

Leakage checker runs when enabled and produces leakage reports.

R-10	Support parallel solution structure	Phase 1

Description:
Ensure initialization pipeline supports multiple task_ids even if num_solutions=1 initially.

Accep

Looping over task_id works; state keys are parameterized correctly.

R-11	Port refinement agent module	Phase 2

Description:
Create sub_agents/refinement/agent.py and prompt.py adapted to the OpenAI runner.

Accep

Module imports cleanly; prompt constants match original intent.

R-12	Port ablation agent	Phase 2

Description:
Implement ablation_agent that generates ablation study scripts.

Accep

ablation_*.py is generated and executed; stdout is captured.

R-13	Port ablation summary agent	Phase 2

Description:
Implement ablation_summary_agent that interprets ablation results.

Accep

Summary is stored in state and fed to init_plan_agent.

R-14	Port init plan agent	Phase 2

Description:

Implement init_plan_agent that proposes a refinement plan and extracts a code block.

Accep

Plan and code_block are parsed and stored.

R-15	Port init plan implement agent	Phase 2

Description:
Implement init_plan_implement_agent that applies the initial plan.

Accep

Improved code is generated, debugged, and scored.

R-16	Port plan refine agent	Phase 2

Description:
Implement plan_refine_agent that proposes alternative improvement plans.

Accep

New plans differ from previous ones and are scored after implementation.

R-17	Port plan implement agent	Phase 2

Description:
Implement plan_implement_agent that applies refined plans.

Accep

Each plan produces executable code and a validation score.

R-18	Wire refinement loops	Phase 2

Description:
Connect inner_loop, outer_loop, and debug loops around refinement agents.

Accep

Refinement repeats outer_loop_round times with inner_loop_round plans each.

R-19	Integrate data leakage checker into refinement	Phase 2

Description:
Attach leakage checks to ablation and plan_implement executions.

Accep

Leakage checker runs during refinement when enabled.

R-20	Port ensemble agent module	Phase 3

Description:
Create sub_agents/ensemble/agent.py and prompt.py adapted to the OpenAI runner.

Accep

Module imports cleanly; prompts match original intent.

R-21	Port init ensemble plan agent	Phase 3

Description:
Implement init_ensemble_plan_agent that proposes how to combine solutions.

Accep

Plan is stored and ready for implementation.

R-22	Port init ensemble plan implement agent	Phase 3

Description:
Implement init_ensemble_plan_implement_agent that builds the initial ensemble.

Accep

ensemble_code_0.py is generated, debugged, and scored.

R-23	Port ensemble plan refine agent	Phase 3

Description:
Implement ensemble_plan_refine_agent that proposes improved ensemble strategies.

Accep

Refined plans are stored and scored.

R-24	Port ensemble plan implement agent	Phase 3

Description:
Implement ensemble_plan_implement_agent that applies refined ensemble plans.

Accep

Each ensemble iteration produces executable code and a validation score.

R-25	Wire ensemble loop	Phase 3

Description:
Connect ensemble refinement loop.

Accep

Loop runs ensemble_loop_round times.

R-26	Integrate data leakage checker into ensemble	Phase 3

Description:
Attach leakage checks to ensemble plan implementation.

Accep

Leakage checker runs during ensemble when enabled.

R-27	Strengthen submission agent	Phase 4

Description:
Update submission_agent to select the best refined solution or ensemble output.

Accep

Best code is chosen correctly; final_solution.py is generated.

R-28	Ensure submission.csv generation	Phase 4

Description:
Verify final_solution.py trains on full train and writes submission.csv.

Accep

submission.csv exists with the expected shape.

R-29	Orchestrate full pipeline in agent.py	Phase 5

Description:
Update top-level agent.py to run initialization, refinement, ensemble, submission in sequence.

Accep

run_pipeline() executes all phases without manual intervention.

R-30	Validate end-to-end on California Housing	Phase 5

Description:
Run the complete pipeline and collect metrics.

Accep

Pipeline finishes; artifacts exist; token/time spend is logged.

8. Risks and mitigations

Model too small for complex JSON

Add robust parsers and retries; log raw responses for debugging.

Excessive token consumption

Use conservative config; monitor total_tokens_spent after each phase.

Data leakage checker too strict

Keep it enabled but cap retries; allow graceful degradation with warnings.

Long execution time

Use small validation subsets inside generated scripts; increase exec_timeout cautiously.

No web search limits model suggestions

Accept reduced retrieval quality in this phase; add search later.

Parallelism with small model multiplies failures

Keep num_solutions=1 until the single-solution path is stable.

9. Suggested timeline

Estimates assume part-time work and iterative debugging with the small university model.

- Phase 0: 1 day
- Phase 1: 3-4 days
- Phase 2: 4-5 days
- Phase 3: 2-3 days
- Phase 4: 1 day
- Phase 5: 2-3 days

Total estimated duration: 2-3 weeks.